MERN Application Infrastructure & Deployment Report

Automated Provisioning via Terraform and Configuration Management via Ansible

Application: TravelMemory (MERN Stack)

Repository: https://github.com/sjoshigit/TravelMemoryTerraformAnsible.git

Execution Environment: WSL2 Ubuntu 22.04 LTS & AWS 2-Tier VPC Architecture

Automation Tools: Terraform v1.5+, Ansible Core 2.14+, PM2, Nginx

---

1. Executive Summary & Objective

This technical report documents the end-to-end automated deployment of the **TravelMemory** MERN application. Infrastructure provisioning is codified using HashiCorp Terraform targeting Amazon Web Services (AWS) and local environments, while configuration orchestration and application lifecycle management are executed using Ansible. The architecture enforces zero-trust network segregation between application and persistence layers.

2. Architecture & Component Interaction

The production topology utilizes a segmented Virtual Private Cloud (VPC) spanning public and private subnets with strict ingress/egress boundaries:

Tier	Component	Network & Port	Security Controls & Routing
Public Tier	Nginx Reverse Proxy & React Frontend	`10.0.1.0/24` | TCP 80, 22	Accepts public HTTP requests; SSH restricted strictly to administrator IP via Security Group.
Application Tier	Node.js / Express API (PM2)	`localhost:3001`	Bound internally on Web instance. Nginx reverse proxies `/api/*` requests directly to port 3001.
Database Tier	MongoDB 7.0 Community Server	`10.0.2.0/24` | TCP 27017	Private subnet; no public IP. Inbound TCP 27017 strictly restricted to Web Server SG. Outbound updates routed via NAT Gateway.

3. Infrastructure as Code (Terraform) Implementation

The infrastructure layer is defined across declarative configurations ensuring reproducible environments:

- **VPC & Subnets (`vpc.tf`):** Created a `10.0.0.0/16` VPC with a public subnet (`10.0.1.0/24`) and private subnet (`10.0.2.0/24`) across availability zone `us-east-1a`.
- **Gateways & Route Tables:** Attached an Internet Gateway (IGW) to the public subnet route table (`0.0.0.0/0`). Deployed an AWS NAT Gateway with an Elastic IP (EIP) in the public subnet, associating it with the private subnet route table for outbound internet access.
- **Compute Provisioning (`ec2.tf`):** Deployed two Canonical Ubuntu 22.04 LTS instances (`t3.micro`) equipped with 20GB gp3 EBS storage. IAM instance profiles attached with `AmazonSSMManagedInstanceCore`.
- **Security Groups (`security_groups.tf`):** Created separate SG primitives enforcing least-privilege egress and ingress rules.

4. Configuration Orchestration (Ansible) & Execution

Ansible modularizes configuration into reusable roles executed sequentially across managed nodes:

- **Common Role (`roles/common`):** Upgrades system packages and installs foundational tooling (`curl` , `git` , `ufw` , `ca-certificates`).
- **Database Role (`roles/mongodb`):** Configured de-armored GPG keys directly in `/etc/apt/keyrings` (resolving Ubuntu 22.04+ apt-key deprecation), configured the official repository, created the `travelmemory` database, and provisioned authenticated access.
- **Web Server Role (`roles/webserver`):** Installed Node.js 18.x, PM2, and Nginx. Cloned the application from `main` branch, rendered environment variables via Jinja2 templates, built the production React bundle, and launched backend via PM2.

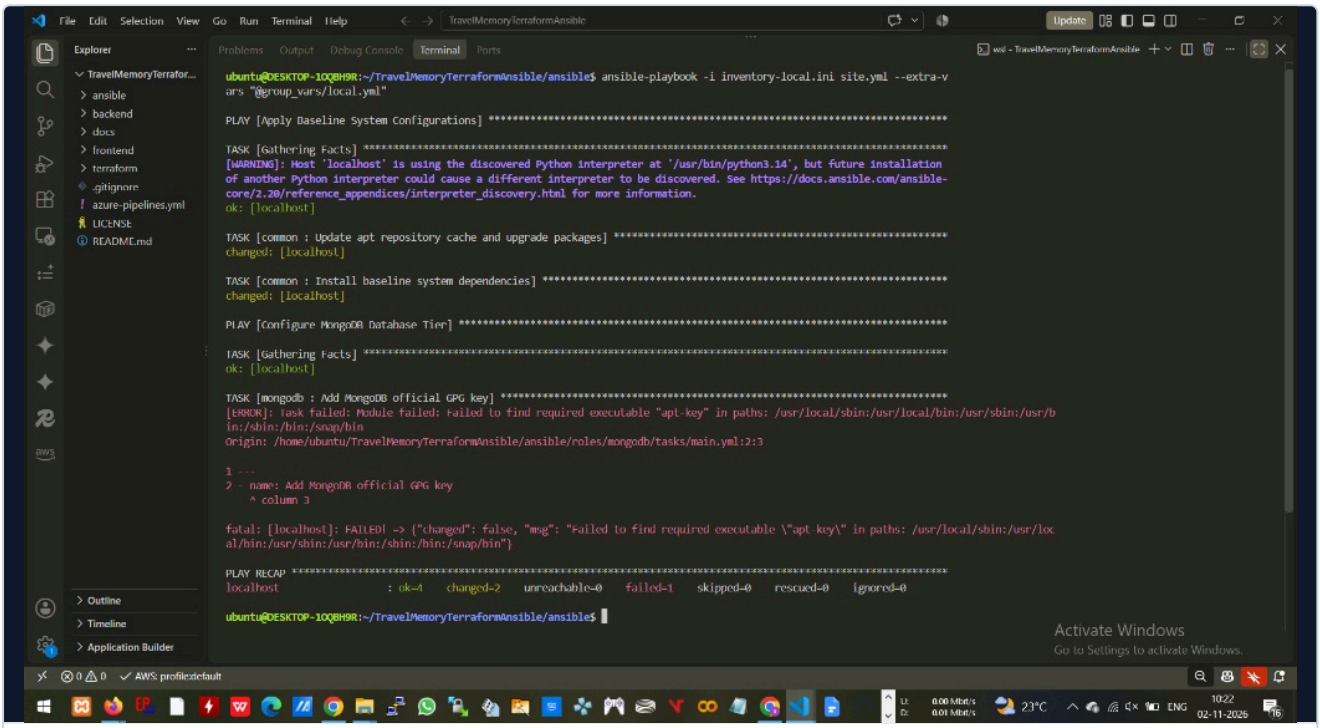


Figure 1: Initial Playbook Debugging — Diagnosing the deprecated `apt-key` executable during MongoDB role setup.

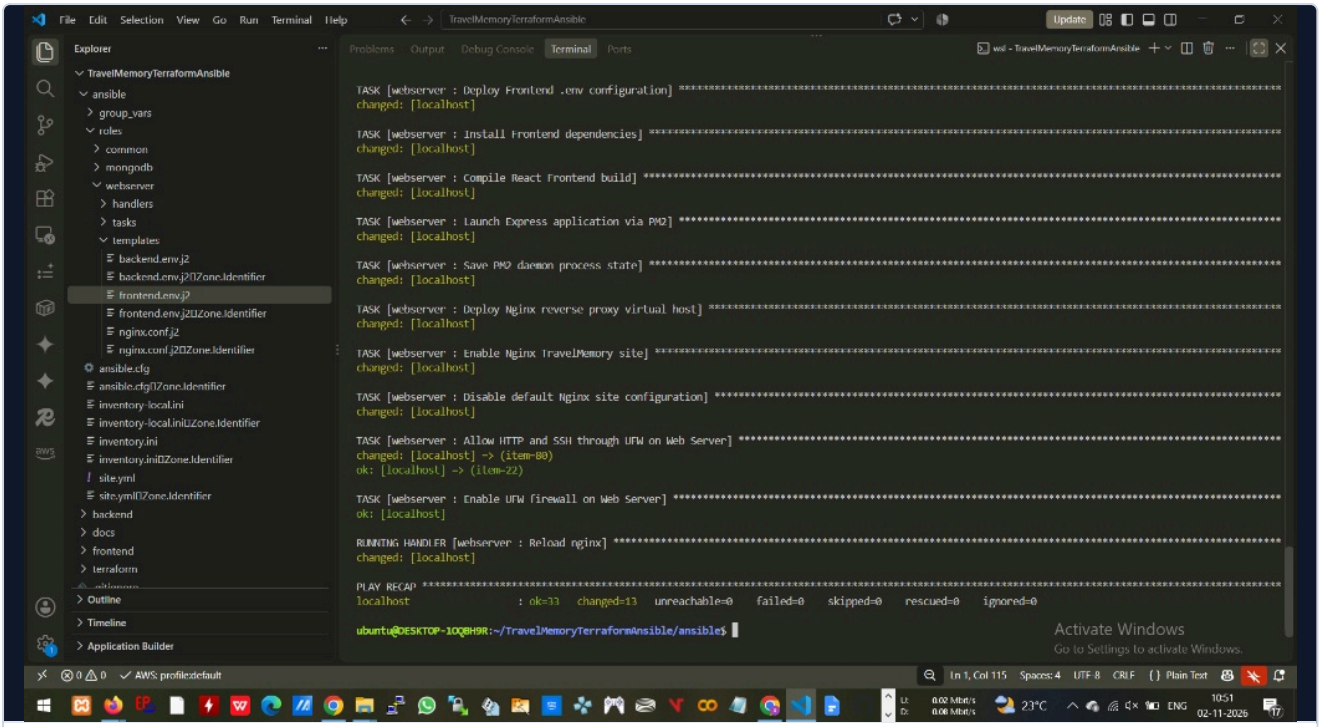


Figure 2: Verified Playbook Execution — Successful run recap showing `ok=33 changed=13 unreachable=0 failed=0`.

5. Service State Verification & Process Management

System services and persistent application processes were validated directly in the runtime environment:

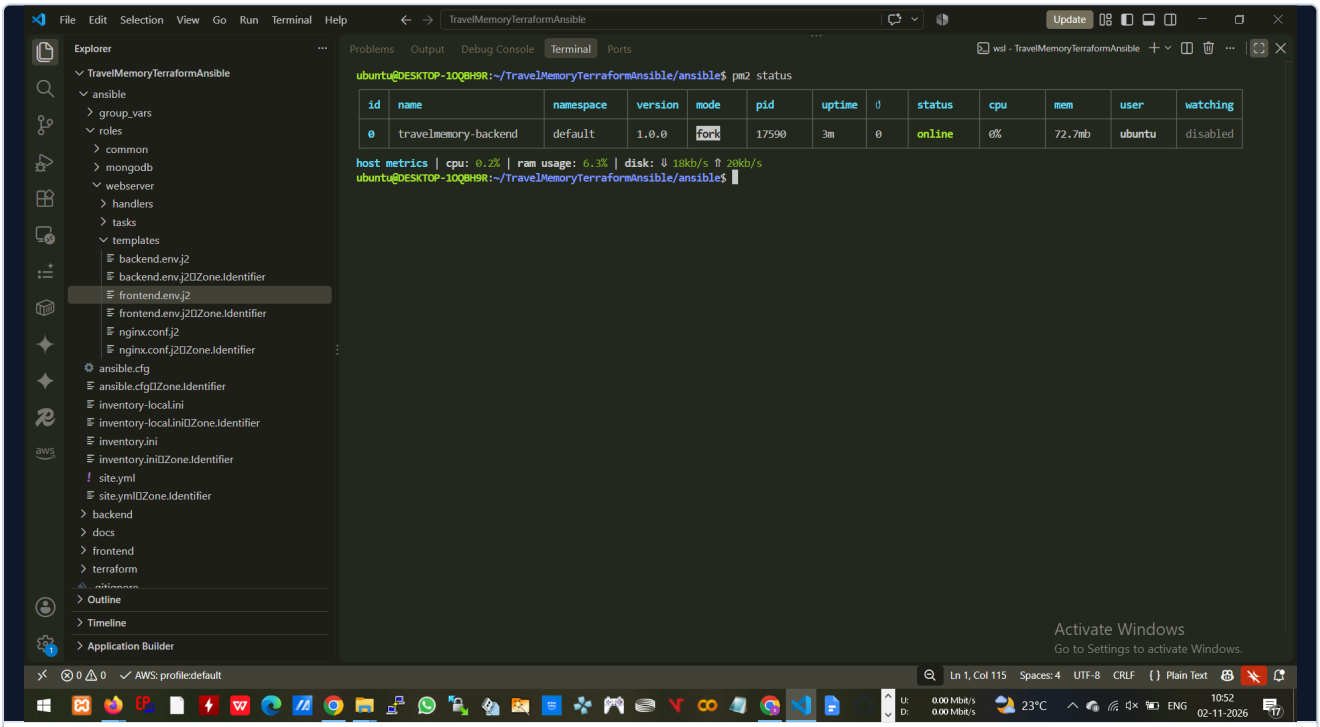


Figure 3: Express Backend in PM2 — `travelmemory-backend` operating in online status on pid 17590.

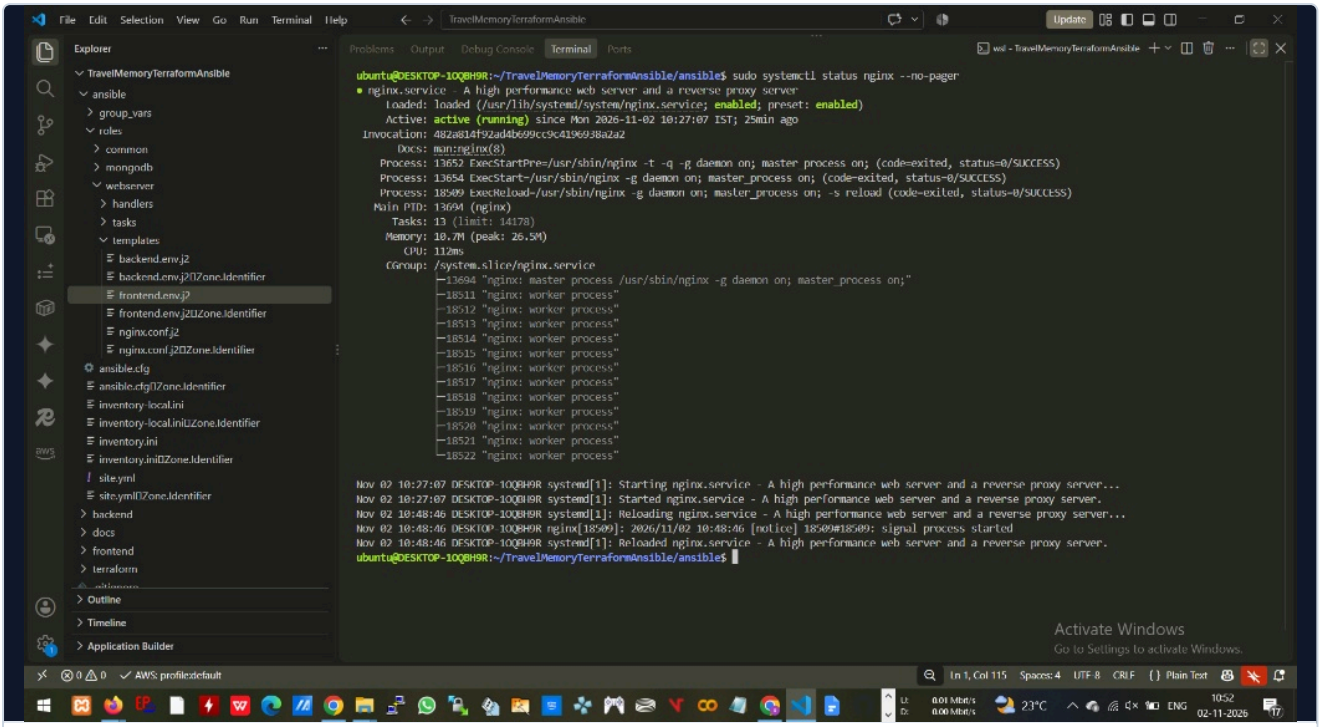


Figure 4: Nginx Reverse Proxy Service — `nginx.service` active (running) serving frontend and routing `/api`.

6. Functional Application Testing

The full MERN loop was verified in the browser. Nginx serves the compiled single-page React frontend on port 80 and proxies experience submission requests to Express and MongoDB.

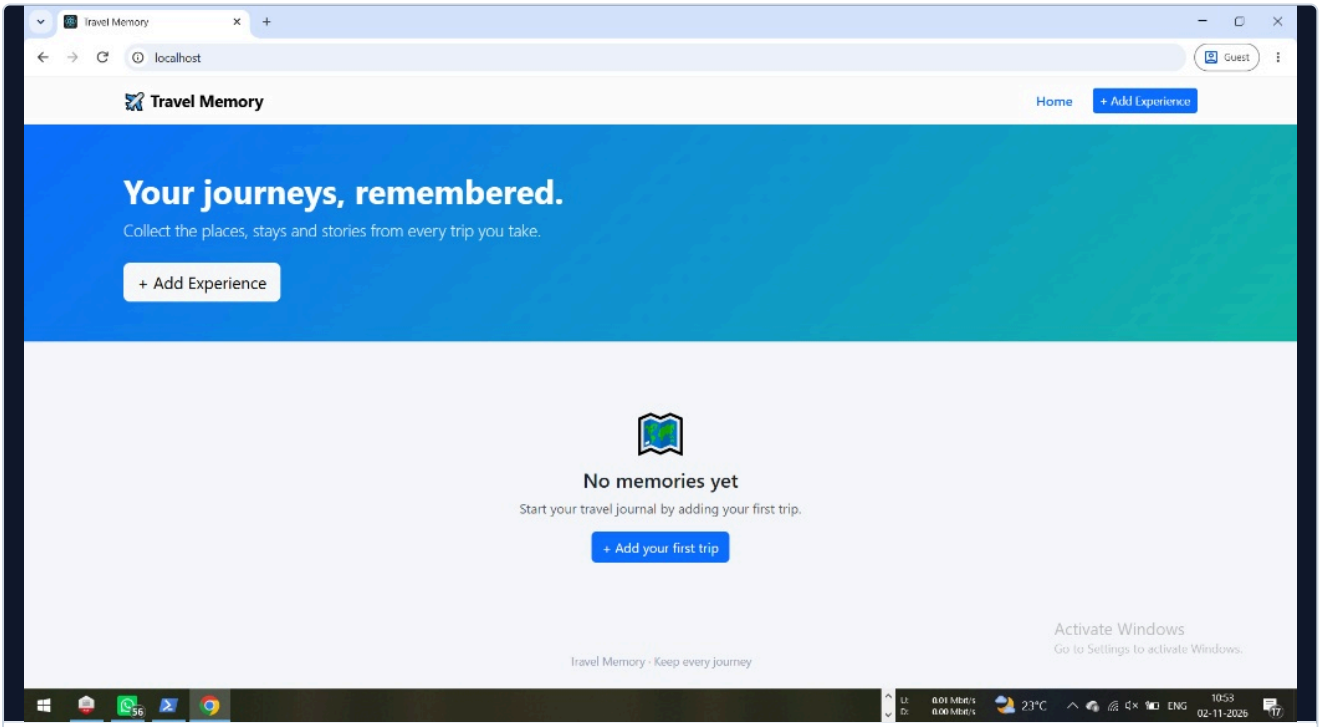


Figure 5: TravelMemory User Interface — React client loaded at `http://localhost` ready for journal entries.

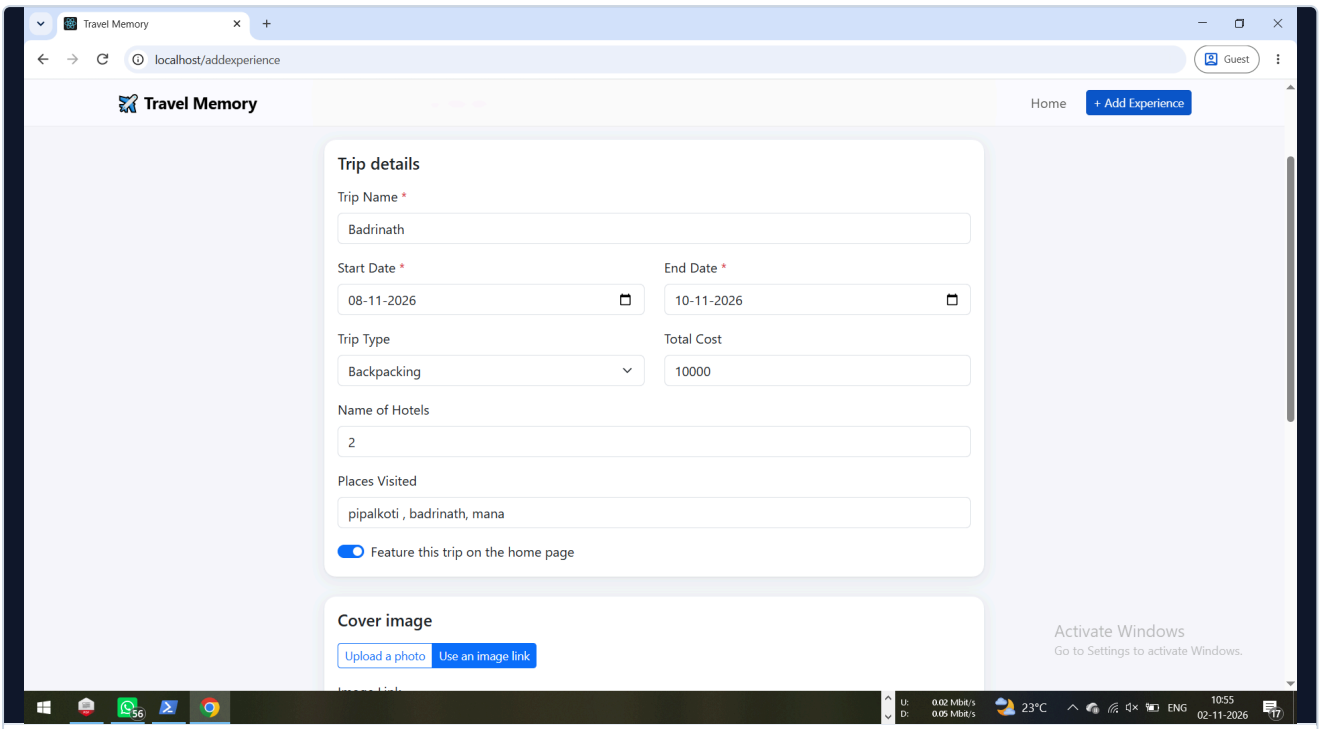


Figure 6: Data Entry & API Communication — Populating new travel memory verifying write operations to MongoDB.

7. Repository & Submission Deliverables

Deliverable	Location in Repository	Description
Terraform Scripts	`/terraform/*.tf`	VPC, subnets, NAT gateway, Security Groups, IAM, and EC2 configurations.
Ansible Playbooks	`/ansible/*`	Roles (`common` , `mongodb` , `webserver`), inventory, site playbook.
Evidence & Artifacts	`/docs/screenshots/`	Screenshots 531 to 537 detailing provisioning logs, PM2 status, Nginx status, and UI.